



Há **62 anos**
reescrevendo histórias!

Sistemas de Informação

**Desenvolvimento de Software
para
Dispositivos Móveis**

Prof. Tiago D. F. Cassimiro
tiago.cassimiro@unifeb.edu.br
2026/2

Centro Universitário da Fundação Educacional de Barretos



Há **62 anos**
reescrevendo histórias!

Aula 03: Android — Manipulação de Eventos e Técnicas de Scroll

Dominando a interatividade e performance em listas dinâmicas



Há **62 anos**
reescrevendo histórias!

Ambiente de Desenvolvimento: Android Studio

Linguagem Utilizada: Kotlin, linguagem oficialmente recomendada para o desenvolvimento moderno de aplicações Android.



Há **62 anos**
reescrevendo histórias!

Objetivos da Aula

- Compreender o conceito de eventos em aplicações Android;
- Identificar os principais tipos de eventos utilizados no desenvolvimento mobile;
- Implementar eventos de clique e interação com componentes;
- Utilizar listeners para capturar ações do usuário;
- Compreender o funcionamento da rolagem de conteúdo;
- Implementar técnicas de scroll utilizando `ScrollView`;
- Conhecer o `RecyclerView` para listas com grande quantidade de dados;
- Desenvolver pequenas aplicações utilizando eventos e técnicas de rolagem.



Há **62 anos**
reescrevendo histórias!

Oque é um evento

Uma aplicação mobile é baseada principalmente na **interação entre o usuário e a interface.**

Quando um usuário toca em um botão, seleciona um item, desliza a tela ou preenche um campo de texto, ele está gerando um **evento.**



Há **62** anos
reescrevendo histórias!

Exemplos de eventos

- . Clique em um botão;
- . Toque na tela;
- . Pressionamento longo;
- . Alteração de texto;
- . Seleção de item;
- . Rolagem vertical;
- . Rolagem horizontal.



Há **62** anos
reescrevendo histórias!

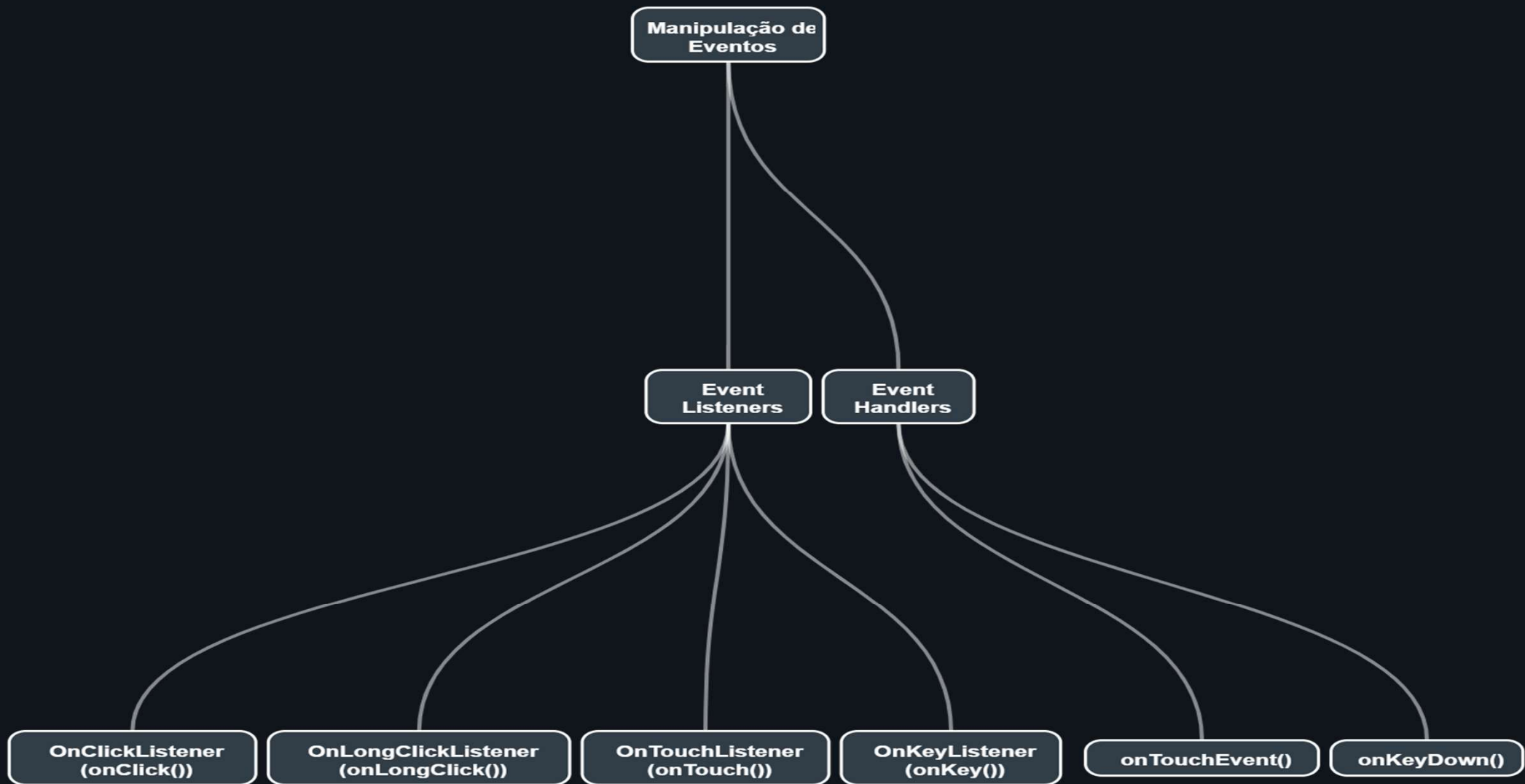
Eventos Android

O Android utiliza mecanismos chamados **listeners**, que ficam responsáveis por "escutar" essas ações e executar determinados comandos.

Manipulação de eventos

- Manipulação de Eventos: Listeners e MotionEvent
- Componentes de Scroll: ScrollView vs. RecyclerView
- Anatomia do RecyclerView: Adapter, ViewHolder, LayoutManager
- Scrolling Aninhado e Resolução de Conflitos
- Performance e UX em Aplicações Android

Manipulação de Eventos no Android



Manipulação de Eventos no Android

A manipulação de eventos é crucial para a interatividade em aplicações Android, baseando-se em dois pilares principais: **Event Listeners** e **Event Handlers**. Enquanto os Listeners são interfaces com métodos de *callback* acionados pelo framework, os Handlers são métodos que podem ser sobrescritos diretamente em classes customizadas que estendem `View`, oferecendo controle mais granular sobre interações específicas.

Clicks e Gestos Longos



Emojis e balões de chat: feedback visual para interações do usuário.

- **OnClickListener**: Acionado após um toque rápido e liberação. Em Kotlin, lambdas: `view.setOnClickListener{...}`.
- **OnLongClickListener**: Acionado ao manter a pressão (> 500ms). Retorna um booleano (true se consumido).
- **Feedback Tátil**: Importante para UX, utilize feedback visual (Ripple Effect) ou háptico.

Anatomia do MotionEvent

Este objeto detalha o estado do toque e as ações realizadas.

Ação	Descrição
ACTION_DOWN	Primeiro dedo encosta na tela (início do gesto).
ACTION_MOVE	Dedo se desloca pela superfície da tela.
ACTION_UP	Dedo é levantado (final do gesto).
ACTION_CANCEL	Gesto interrompido (ex: scroll de container pai).

Containers de Scroll

Quando o conteúdo excede o tamanho da tela, utilizamos o **ScrollView** para rolagem vertical e o **HorizontalScrollView** para rolagem horizontal. Ambos permitem a manipulação de eventos de *scroll* e toques. Possuem uma *restrição crucial*: podem conter apenas **um** filho direto (geralmente um `LinearLayout` ou `ConstraintLayout` para agrupar múltiplos elementos).

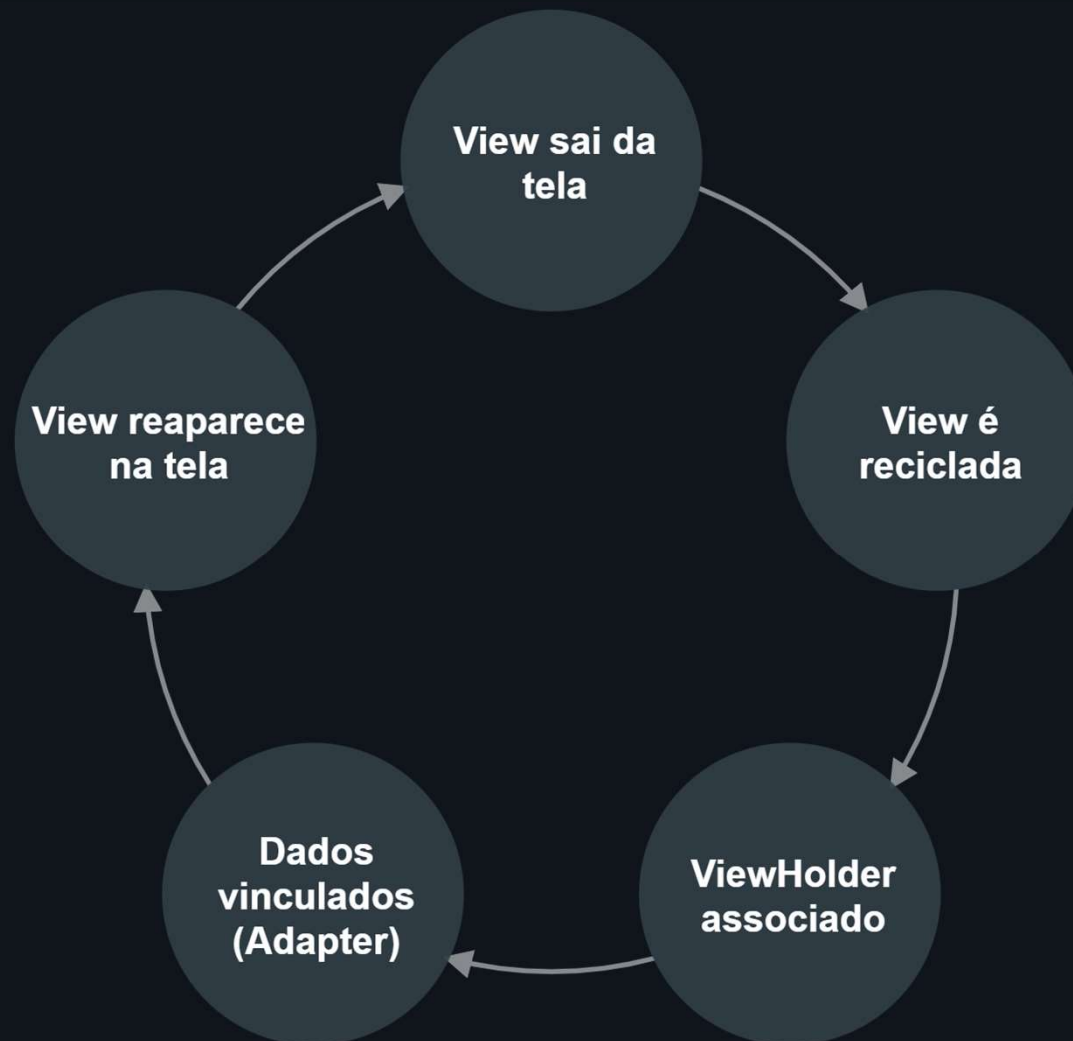
Limitações e Desempenho

Apesar de sua simplicidade, ScrollView e HorizontalScrollView carregam *todos* os componentes na memória simultaneamente. Essa característica impacta diretamente na performance, não sendo recomendados para listas grandes ou dinâmicas. O alto consumo de recursos pode causar quedas significativas nos **FPS** (*Frames Per Second*), prejudicando a fluidez da interface.

RecyclerView: Eficiência Avançada

Como vimos, o `RecyclerView` é a solução padrão para listas grandes ou coleções de dados dinâmicos, superando as limitações do `ScrollView`. Ele atinge **alta performance** por meio da *reutilização de Views*, onde Views que saem da tela são recicladas e preenchidas com novos dados. Sua arquitetura é desacoplada, delegando responsabilidades para componentes específicos: o **Adapter** (conecta dados à View), o **ViewHolder** (cacheia referências de Views filhas) e o **LayoutManager** (define a disposição dos itens).

RecyclerView: Eficiência Avançada



Componentes Essenciais do RecyclerView



LayoutManager

Determina a **organização visual** dos itens na lista.

Exemplos incluem

`LinearLayoutManager` para listas lineares,

`GridLayoutManager` para arranjos em grade, e

`StaggeredGridLayoutManager` para layouts de grade com itens de diferentes alturas.



ItemDecoration

Responsável por desenhar **divisores**, margens ou offsets entre os itens. Atua na camada de desenho, sem modificar o layout de cada item individualmente, permitindo personalizações visuais.

Componentes Essenciais do RecyclerView



ItemAnimator

Define as **animações** para as ações de adicionar, remover ou reordenar itens na lista. O `DefaultItemAnimator` provê animações padrão que garantem uma experiência de usuário fluida e responsiva.

Ciclo de Vida do Adapter

1

Declaração do ViewHolder

Crie uma classe interna que estende `RecyclerView.ViewHolder`. Ela mapeia os IDs do layout XML para propriedades, evitando buscas repetitivas por views.

2

Implementação do `onCreateViewHolder()`

Neste método, o layout XML de cada item é *inflado* e uma nova instância do ViewHolder é criada. Este ViewHolder será reciclado para reutilização de views.

3

Implementação do `onBindViewHolder()`

Associa os dados da fonte (ex: uma `List`) às views específicas dentro do ViewHolder, utilizando a `position` atual do item para exibir as informações corretas.

4

Implementação do `getItemCount()`

Este método informa ao RecyclerView o tamanho total da fonte de dados. É crucial para que o RecyclerView possa planejar eficientemente a rolagem e a exibição dos itens.

Scrolling Aninhado e Conflitos

O **Scrolling Aninhado** (Nested Scrolling) permite que múltiplas *views* roláveis interajam de forma coordenada, crucial para interfaces complexas. O **NestedScrollView** é uma versão do `ScrollView` que implementa esse protocolo, e o **CoordinatorLayout** atua como um container pai que orquestra interações mais elaboradas, como o comportamento de barras de ferramentas durante a rolagem. Contudo, é fundamental evitar colocar um `RecyclerView` dentro de um `ScrollView` comum, pois isso pode comprometer a eficiência de reciclagem de memória do `RecyclerView` e causar problemas de rolagem.

No contexto do desenvolvimento Android com ****RecyclerView****, que é fundamental para exibir listas eficientes, qual é o principal benefício do padrão ***ViewHolder***?

A Evitar chamadas repetitivas ao método `findViewById()`, melhorando a performance de rolagem.

B Gerenciar automaticamente a conexão com bancos de dados remotos.

C Permitir que a lista aceite diferentes tipos de dados simultaneamente.

D Garantir que a lista sempre role na horizontal e vertical ao mesmo tempo.

No contexto do desenvolvimento Android com `**RecyclerView**`, que é fundamental para exibir listas eficientes, qual é o principal benefício do padrão `*ViewHolder*`?



Evitar chamadas repetitivas ao método `findViewById()`, melhorando a performance de rolagem.



Gerenciar automaticamente a conexão com bancos de dados remotos.



Permitir que a lista aceite diferentes tipos de dados simultaneamente.



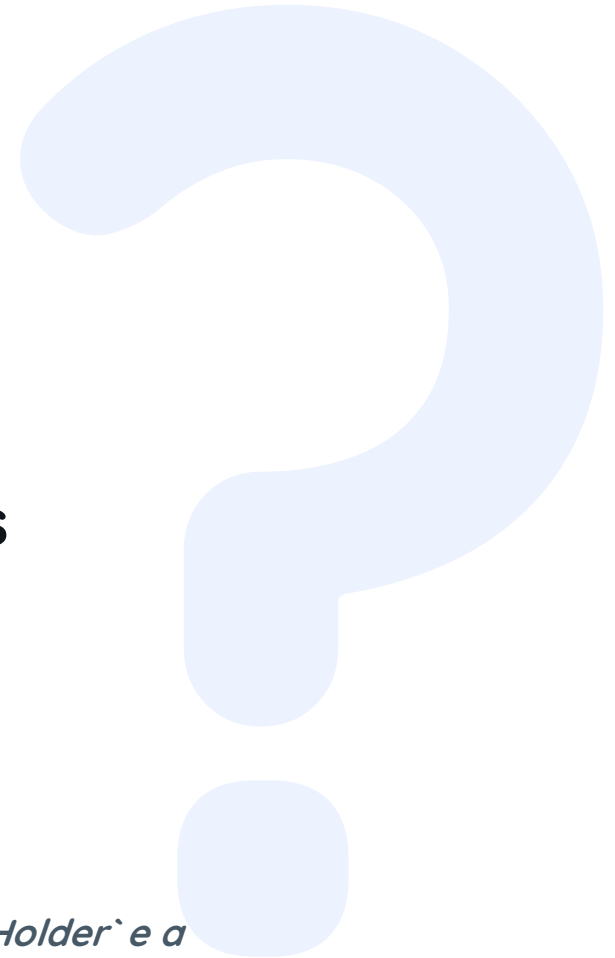
Garantir que a lista sempre role na horizontal e vertical ao mesmo tempo.



Discuta com o colega ao lado

Como podemos equilibrar a complexidade visual e a fluidez da aplicação Android ao manipular listas, considerando o impacto de ferramentas como `DiffUtil` e o local de processamento de dados?

Reflitam sobre os riscos do processamento pesado em `onBindViewHolder` e a percepção de performance do usuário em listas extensas.



Conclusão

- Event Listeners (OnClickListener, onTouchListener) e Event Handlers (onTouchEvent) são a base da interação no Android, capturando ações do usuário.
- O MotionEvent detalha toques (ACTION_DOWN, ACTION_MOVE, ACTION_UP), fornecendo coordenadas para gestos complexos e precisos.
- RecyclerView otimiza listas grandes reusando Views (ViewHolder) e delegando layout (LayoutManager) para alta performance de rolagem.
- Scrolling aninhado com NestedScrollView e CoordinatorLayout gerencia interações complexas, evitando conflitos de rolagem em UIs sofisticadas.



Há **62 anos**
reescrevendo histórias!

Conclusão

O Android utiliza mecanismos chamados **listeners**, que ficam responsáveis por "escutar" essas ações e executar determinados comandos.



Há **62 anos**
reescrevendo histórias!

Referencias

- DEITEL, Paul; DEITEL, Harvey; DEITEL, Abbey. **Android: como programar**. 2. ed. Porto Alegre: Bookman, 2015.
- GRIFFITHS, Dawn; GRIFFITHS, David. **Use a cabeça! Desenvolvimento Android**. Rio de Janeiro: Alta Books, 2019.
- MEIER, Reto. **Professional Android 4 application development**. Indianapolis: Wiley, 2012.
- PHILLIPS, Bill; STEWART, Chris; MARSICANO, Kristin. **Android programming: the Big Nerd Ranch guide**. 4. ed. Atlanta: Big Nerd Ranch, 2019.
- GOOGLE. **Android Developers: developer guides**. Mountain View: Google. Disponível em: Android Developers. Acesso em: 1 set. 2026.
- GOOGLE. **Kotlin and Android**. Mountain View: Google. Disponível em: Android Developers. Acesso em: 1 set. 2026.
- GOOGLE. **ScrollView — Android Developers**. Mountain View: Google. Disponível em: Android Developers. Acesso em: 1 set. 2026.
- JETBRAINS. **Kotlin documentation**. Praga: JetBrains. Disponível em: Kotlin Documentation. Acesso em: 1 set. 2026.